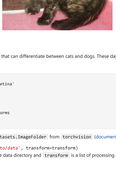
# Loading Image Data

So far we've been working with fairly artificial datasets that you wouldn't typically be using in real projects. Instead, you'll likely be dealing with full-sized images like you'd get from smart phone cameras. In this notebook, we'll look at how to load images and use them to train neural networks.

We'll be using a dataset of cat and dog photos available from Kaggle. Here are a couple example images:



We'll use this dataset to train a neural network that can differentiate between cats and dogs. These days it doesn't seem like a big accomplishment, but five years ago it was a serious challenge for computer vision systems.

In [1]:
```
%matplotlib inline
%config InlineBackend.figure_format = 'retina'

import matplotlib.pyplot as plt

import torch
from torchvision import datasets, transforms

import helper
```

The easiest way to load image data is with `datasets.ImageFolder` from `torchvision` (documentation). In general you'll use `ImageFolder` like so:

```
dataset = datasets.ImageFolder('path/to/data', transform=transform)
```

where `'path/to/data'` is the file path to the data directory and `transform` is a list of processing steps built with the `transforms` module from `torchvision`. ImageFolder expects the files and directories to be constructed like so:

```
root/dog/xxx.png
root/dog/xxy.png
root/dog/xxz.png

root/cat/123.png
root/cat/nsdf3.png
root/cat/asd932_.png
```

where each class has its own directory (`cat` and `dog`) for the images. The images are then labeled with the class taken from the directory name. So here, the image `123.png` would be loaded with the class label `cat`. You can download the dataset already structured like this from here. I've also split it into a training set and test set.

## Transforms

When you load in the data with `ImageFolder`, you'll need to define some transforms. For example, the images are different sizes but we'll need them to all be the same size for training. You can either resize them with `transforms.Resize()` or crop with `transforms.CenterCrop()`, `transforms.RandomResizedCrop()`, etc. We'll also need to convert the images to PyTorch tensors with `transforms.ToTensor()`. Typically you'll combine these transforms into a pipeline with `transforms.Compose()`, which accepts a list of transforms and runs them in sequence. It looks something like this to scale, then crop, then convert to a tensor:

```
transform = transforms.Compose([transforms.Resize(255),
                                transforms.CenterCrop(224),
                                transforms.ToTensor()])
```

There are plenty of transforms available, I'll cover more in a bit and you can read through the documentation.

## Data Loaders

With the `ImageFolder` loaded, you have to pass it to a `DataLoader`. The `DataLoader` takes a dataset (such as you would get from `ImageFolder`) and returns batches of images and the corresponding labels. You can set various parameters like the batch size and if the data is shuffled after each epoch.

```
dataloader = torch.utils.data.DataLoader(dataset, batch_size=32, shuffle=True)
```

Here `DataLoader` is a generator. To get data out of it, you need to loop through it or convert it to an iterator and call `next()`.

```
# Looping through it, get a batch on each loop
for images, labels in dataloader:
    pass

# Get one batch
images, labels = next(iter(dataloader))
```

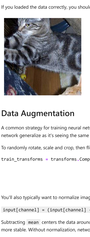
> **Exercise:** Load images from the `Cat_Dog_data/train` folder, define a few transforms, then build the dataloader.

In [10]:
```
data_dir = 'C:\Users\aryav\Desktop\Dray\Cat_Dog_data\train'

transform = transforms.Compose([transforms.Resize(255),
                                transforms.CenterCrop(224),
                                transforms.ToTensor()])
dataset = datasets.ImageFolder(data_dir, transform=transform)
dataloader = torch.utils.data.DataLoader(dataset, batch_size=32, shuffle=True)
```

In [13]:
```
# Run this to test your data loader
images, labels = next(iter(dataloader))
helper.imshow(images[0], normalize=False)
```

Out[13]:
```
<Axes: >
```

If you loaded the data correctly, you should see something like this (your image will be different):



## Data Augmentation

A common strategy for training neural networks is to introduce randomness in the input data itself. For example, you can randomly rotate, mirror, scale, and/or crop your images during training. This will help your network generalize as it's seeing the same images but in different locations, with different sizes, in different orientations, etc.

To randomly rotate, scale and crop, then flip your images you would define your transforms like this:

```
train_transforms = transforms.Compose([transforms.RandomRotation(30),
                                       transforms.RandomResizedCrop(224),
                                       transforms.RandomHorizontalFlip(),
                                       transforms.ToTensor(),
                                       transforms.Normalize([0.5, 0.5, 0.5],
                                                            [0.5, 0.5, 0.5])])
```

You'll also typically want to normalize images with `transforms.Normalize`. You pass in a list of means and list of standard deviations, then the color channels are normalized like so

```
input[channel] = (input[channel] - mean[channel]) / std[channel]
```

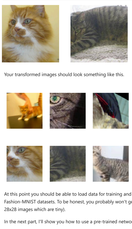
Subtracting `mean` centers the data around zero and dividing by `std` squishes the values to be between -1 and 1. Normalizing helps keep the network work weights near zero which in turn makes backpropagation more stable. Without normalization, networks will tend to fail to learn.

You can find a list of all the available transforms here. When you're testing however, you'll want to use images that aren't altered (except you'll need to normalize the same way). So, for validation/test images, you'll typically just resize and crop.

> **Exercise:** Define transforms for training data and testing data below. Leave off normalization for now.

In [12]:
```
data_dir = 'C:\Users\aryav\Desktop\Dray\Cat_Dog_data'

# TODO: Define transforms for the training data and testing data
train_transforms = transforms.Compose([transforms.RandomRotation(30),
                                       transforms.RandomResizedCrop(224),
                                       transforms.RandomHorizontalFlip(),
                                       transforms.ToTensor()])

test_transforms = transforms.Compose([transforms.Resize(255),
                                      transforms.CenterCrop(224),
                                      transforms.ToTensor()])


# Pass transforms in here, then run the next cell to see how the transforms look
train_data = datasets.ImageFolder(data_dir + '/train', transform=train_transforms)
test_data = datasets.ImageFolder(data_dir + '/test', transform=test_transforms)

trainloader = torch.utils.data.DataLoader(train_data, batch_size=32)
testloader = torch.utils.data.DataLoader(test_data, batch_size=32)
```

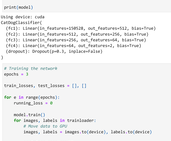
In [13]:
```
# change this to the trainloader or testloader
data_iter = iter(testloader)

images, labels = next(data_iter)
fig, axes = plt.subplots(figsize=(10,4), ncols=4)
for ii in range(4):
    ax = axes[ii]
    helper.imshow(images[ii], ax=ax, normalize=False)
```



Your transformed images should look something like this.

Training examples:

Testing examples:



At this point you should be able to load data for training and testing. Now, you should try building a network that can classify cats vs dogs. This is quite a bit more complicated than before with the MNIST and Fashion MNIST datasets. To be honest, you probably won't get it to work with a fully connected network, no matter how deep. These images have three color channels and at a higher resolution (so far you've seen 28x28 images which are tiny).

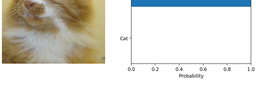
In the next part, I'll show you how to use a pre-trained network to build a model that can actually solve this problem.

In [14]:
```
# Optional TODO: Attempt to build a network to classify cats vs dogs from this dataset
from torch import nn, optim
import torch.nn.functional as F

# Check for GPU availability
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")

# Define the classifier using a simple fully connected network
# Note: This is rarely challenging for FC networks due to high resolution (224x224x3 = 150528 inputs)
class CatDogClassifier(nn.Module):
    def __init__(self):
        super().__init__()
        # Input: 224x224x3 = 150528
        self.fc1 = nn.Linear(224*224*3, 512)
        self.fc2 = nn.Linear(512, 256)
        self.fc3 = nn.Linear(256, 64)
        self.fc4 = nn.Linear(64, 2)  # 2 classes: cat and dog

        self.dropout = nn.Dropout(p=0.3)

    def forward(self, x):
        # Flatten the image
        x = x.view(x.shape[0], -1)

        x = self.dropout(F.relu(self.fc1(x)))
        x = self.dropout(F.relu(self.fc2(x)))
        x = self.dropout(F.relu(self.fc3(x)))
        x = F.log_softmax(self.fc4(x), dim=1)

        return x

# Create model, criterion and optimizer
model = CatDogClassifier()
model.to(device)  # Move model to GPU
criterion = nn.NLLLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

print(model)
```

```
Using device: cuda
CatDogClassifier(
  (fc1): Linear(in_features=150528, out_features=512, bias=True)
  (fc2): Linear(in_features=512, out_features=256, bias=True)
  (fc3): Linear(in_features=256, out_features=64, bias=True)
  (fc4): Linear(in_features=64, out_features=2, bias=True)
  (dropout): Dropout(p=0.3, inplace=False)
)
```

In [15]:
```
# Training the network
epochs = 3

train_losses, test_losses = [], []

for e in range(epochs):
    running_loss = 0

    model.train()
    for images, labels in trainloader:
        # Move data to GPU
        images, labels = images.to(device), labels.to(device)

        optimizer.zero_grad()

        log_ps = model(images)
        loss = criterion(log_ps, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item()

    # Validation pass
    test_loss = 0
    accuracy = 0

    model.eval()
    with torch.no_grad():
        for images, labels in testloader:
            # Move data to GPU
            images, labels = images.to(device), labels.to(device)

            log_ps = model(images)
            test_loss += criterion(log_ps, labels).item()

            ps = torch.exp(log_ps)
            top_p, top_class = ps.topk(1, dim=1)
            equals = top_class == labels.view(*top_class.shape)
            accuracy += torch.mean(equals.type(torch.FloatTensor)).item()

    train_losses.append(running_loss / len(trainloader))
    test_losses.append(test_loss / len(testloader))

    print(f"Epoch {e+1}/{epochs}.. "
          f"Train loss: {train_losses[-1]:.3f}.. "
          f"Test loss: {test_losses[-1]:.3f}.. "
          f"Test accuracy: {accuracy/len(testloader):.3f}")
```

```
Epoch 1/3.. Train loss: 1.729.. Test loss: 791.142.. Test accuracy: 0.506
Epoch 2/3.. Train loss: 0.284.. Test loss: 318.935.. Test accuracy: 0.506
Epoch 3/3.. Train loss: 3.989.. Test loss: 332.163.. Test accuracy: 0.506
```

In [16]:
```
# Plot training and validation losses
plt.plot(train_losses, label='Training loss')
plt.plot(test_losses, label='Validation loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend(frameon=False)
plt.title('Cat vs Dog Classifier Training')
plt.show()
```

In [23]:
```
# Test the model with a sample image
model.eval()

images, labels = next(iter(testloader))
img = images[0]

# Forward pass (move to GPU for inference, then back to CPU for plotting)
with torch.no_grad():
    log_ps = model(img.unsqueeze(0).to(device))
    ps = torch.exp(log_ps).cpu()  # Move back to CPU for numpy operations

# Get class names and prediction
classes = ['Cat', 'Dog']
top_p, top_class = ps.topk(1, dim=1)

# Display result
fig, (ax1, ax2) = plt.subplots(figsize=(8, 4), ncols=2)

# Show image (already on CPU)
ax1.imshow(img.numpy().transpose((1, 2, 0)))
ax1.set_title("True: {classes[labels[0]]}")
ax1.axis('off')

# Show probabilities
ax2.barh(classes, ps.numpy()[0].squeeze())
ax2.set_xlim(0, 1)
ax2.set_title(f'Predicted: {classes[top_class.item()]} ({top_p.item():.2%})')
ax2.set_xlabel('Probability')

plt.tight_layout()
plt.show()
```